

Servlet Life Cycle Analysis

Comprehensive technical deep dive into Servlet Architecture,
Concurrency, and Optimization

CSA4305-Internet programming

Presented by:

G.Abhinay

192373062

The Servlet Lifecycle Architecture

Container Managed Model (IoC)

The Web Container controls class loading, instantiation, configurations, thread-to-request dispatching, and de-allocation. Developers focus solely on business logic, leaving network protocols and socket management to the container.

Singleton Architecture Pattern

To conserve JVM heap memory and execution overhead, the container instantiates exactly one instance of a servlet class. This single instance is kept in memory and shared globally to service all concurrent requests.

The Three Lifecycle Milestones

1. `init()`: Invoked once to allocate configurations and resources.
2. `service()`: Invoked multiple times concurrently to process incoming HTTP requests.
3. `destroy()`: Invoked once to shut down resource footprints.

Phase 1: Deep Dive on init()

Config Injection (ServletConfig vs ServletContext)

The container injects a `ServletConfig` object containing servlet-specific parameters (`init-param`), and links it to the broader `ServletContext` representing the entire web application scope.

Heavy Setup Tasks & Resource Allocation

Used to perform expensive, one-time setups: establishing database connection pools, initializing file buffers, configuring cache maps, and connecting to third-party APIs.

Eager vs. Lazy Initialization Strategies

- **Lazy Loading:** Servlet is loaded only when the first client requests it. This introduces a significant response penalty (first-request lag) to the initial client.
- **Eager Loading:** Using `load-on-startup` (values ≥ 0) forces initialization during server deployment, moving setup costs away from active clients.

Phase 2: Deep Dive on service()

The Dispatcher Workflow

Whenever an HTTP request maps to the servlet, the container allocates a thread, executes `service()`, and parses the `ServletRequest` and `ServletResponse` objects.

Routing in HttpServlet

The `service()` method acts as a traffic router. It reads the incoming request verb (`getMethod()`) and forwards execution to the appropriate helper method: `doGet()`, `doPost()`, `doPut()`, or `doDelete()`.

Multi-Threaded Execution Environment

The container maintains a thread pool (e.g., Tomcat execution threads). Multiple clients hitting the same URL trigger concurrent `service()` executions on the SAME servlet instance. The servlet **MUST** be thread-safe.

Execution Flow Sequence details

1. Server URL Mapping Resolution

The client issues a request. The container scans deployment configurations (XML/annotations) to locate the target servlet mapping.

2. Instantiation & Setup

If missing, the container loads the class using ClassLoader reflection, instantiates it, and invokes init().

3. Thread Mapping & dispatch

A thread is pulled from the container's pool to execute service(). The thread handles request parsing and response writing.

4. Execution & Response

service() maps the request to doGet/doPost. The servlet generates dynamic response data and writes it back to the client socket.

5. destruction Phase

When stopping the web context, the container runs destroy() and clears the instance for JVM garbage collection.

Phase 3: Deep Dive on destroy()

Release of Allocated System Resources

Failing to release resources leads to system socket leaks and memory leaks. The `destroy()` method is used to gracefully close database connections, shutdown thread pools, and release file system locks.

Grace Period & Active Threads

Before calling `destroy()`, the container stops dispatching incoming requests. It waits a configurable grace period for currently executing `service()` threads to complete their execution.

Garbage Collection & Lifecycle Termination

Following the completion of `destroy()`, the container releases its reference to the servlet. The servlet instance is marked eligible for JVM garbage collection.

Performance & Concurrency Analysis

1. Thread Safety: The Stateless Mandate

Because a single servlet serves hundreds of concurrent client threads, avoid using instance variables to store client-specific states. Use local stack variables inside doGet/doPost to ensure thread safety without synchronization locks.

2. Avoid Service-Level Connection Overhead

Never instantiate database connections or open files inside doGet() or doPost(). This creates major performance overhead. Always configure shared resource pools inside init() and reuse them.

3. Pre-Warming (Load On Startup)

Set load-on-startup ≥ 0 in WebServlet configuration. This shifts the classloader and initialization execution cost to container boot time, ensuring fast first-user interactions.

Best Practices Summary

- **Leverage Stateless Designs**

Guarantees thread safety and unlocks peak container throughput without CPU synchronization overhead.

- **Use Eager Initialization for High-Availability**

Avoid first-request latency spikes by deploying with load-on-startup configured.

- **Ensure Resource Cleanups in destroy()**

Prevents severe system file-handle and connection-pool resource leaks over time.

- **Delegate Middleware to Servlet Filters**

Offload cross-cutting concerns (authentication, logging, payload compression) to filters, keeping servlets fast and singular in purpose.

THANK YOU